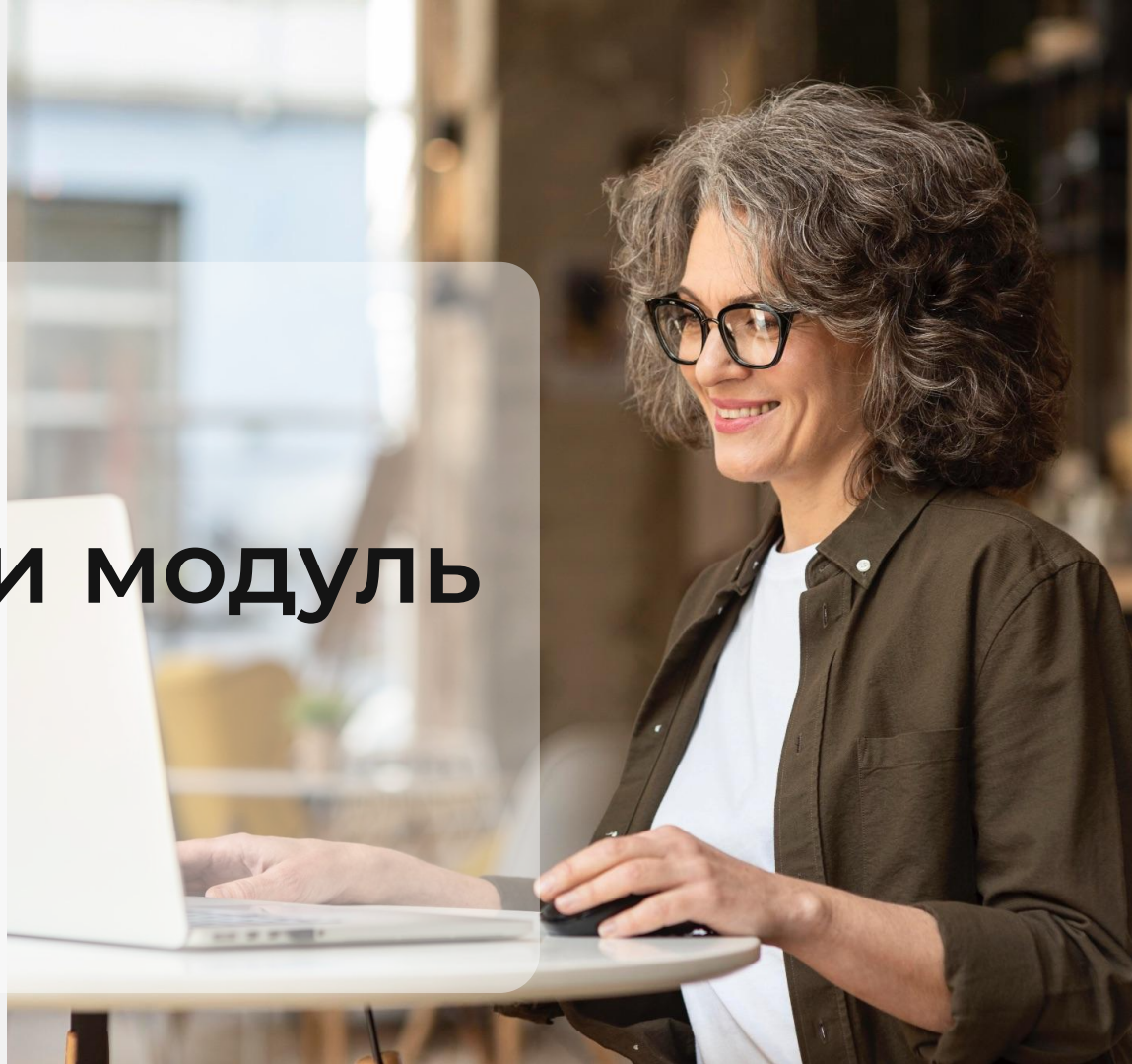


Python

Файлы CSV и модуль statistics



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение



JSON



Модуль json



Сравнение типов Python и JSON



Форматирование JSON



JSONDecodeError



Модуль datetime

План занятия

- Формат CSV
- Работа с CSV в Python
- Недостатки модуля CSV
- Модуль statistics



ОСНОВНОЙ БЛОК





Формат CSV



CSV (Comma-Separated Values)

Это простой текстовый формат для хранения табличных данных. Каждая строка представляет собой одну запись, а значения разделяются запятыми, или другими разделителями

Пример CSV-файла

```
name,age,city  
Alice,30,New York  
Bob,25,London  
Charlie,35,Paris
```

Особенности CSV



Каждая строка файла представляет собой запись



Поля внутри записи разделяются запятыми



Первая строка файла часто содержит заголовки колонок



Поддерживает как числовые, так и текстовые данные, но все трактуются как строковый тип

Использование CSV



Хранение и передача данных между системами



Импорт и экспорт данных в табличных редакторах (например, Excel)



Обмен данными между различными приложениями

Сравнение CSV и JSON

Характеристика	CSV	JSON
Структура данных	Табличная структура с фиксированными колонками	Иерархическая структура с вложенными объектами и массивами
Типы данных	Все данные обычно строки, требуется преобразование	Поддержка различных типов данных (строки, числа, булевы значения, списки, объекты)
Читаемость	Легко читается человеком, особенно для простых данных	Легко читается подготовленным человеком, знакомым с иерархической структурой, но структура может быть громоздкой
Поддержка пропущенных значений	Пропущенные значения обычно представлены как пустые строки	Поддержка <code>null</code> для обозначения отсутствующих значений
Размер файла	Обычно меньше для простых данных	Может быть больше из-за метаданных и структуры

Сравнение CSV и JSON

Характеристика	CSV	JSON
Инструменты для работы	Модуль csv в Python, Excel	Библиотеки json в Python, JavaScript
Производительность при чтении/записи	Обычно быстрее для больших объемов данных	Может быть медленнее из-за необходимости разбора структуры
Использование	Хранение и обмен табличными данными	Хранение и обмен структурированными данными, конфигурации, API
Поддержка вложенных структур	Нет	Да
Стандартизация	Широко используемый стандарт	Широко используемый стандарт



ВОПРОСЫ





Работа с CSV в Python

Важно



Python поддерживает работу с CSV через встроенный модуль csv

Чтение CSV-файла

```
import csv
with open('sales.csv') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```



Reader

Это специальный объект, который
построчно считывает данные из файла и
выдает их в виде списка строк

Разделитель



По умолчанию

По умолчанию в формате CSV используется запятая как разделитель между данными в строке

Указать другой

```
delimiter=';'
```

Запись в CSV-файл

```
with open('output.csv', 'w', newline='') as f:  
    writer = csv.writer(f)  
    writer.writerow(['name', 'age'])  
    writer.writerow(['Alice', 30])
```

Важно



Атрибут `newline=' '` стоит указывать, чтобы в файле не было лишних пустых строк

Метод writerows()



Запись списка строк

```
rows = [
    ['Alice', 30],
    ['Bob', 25],
    ['Charlie', 35]
]
with open('output.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['name', 'age']) #
заголовок
    writer.writerows(rows)           # сразу
все строки
```

Добавление строк в существующий файл

```
with open('sales.csv', 'a', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Orange', 'Rainbow',
                    '2025-07-04', '110', '2.00'])
```

Работа со словарями



Можно считывать данные из csv-файла в словарь. В этом случае заголовки столбцов станут ключами значений в каждой строке

Пример считывания в словарь

```
with open('sales.csv') as f:  
    reader = csv.DictReader(f)  
    for row in reader:  
        print(row)
```

Запись словаря в CSV-файл



Пример

```
with open('output.csv', 'w',  
        newline='') as f:  
    writer = csv.DictWriter(f,  
        fieldnames=['name', 'age'])  
    writer.writeheader()  
    writer.writerow({'age': 30, 'name':  
        'Alice'})
```

Пояснения

- Аргумент `fieldnames` обязателен: он задаёт порядок столбцов и их названия
- `writeheader()` необходим для записи строки с заголовком

Запись в файл списка словарей



Пример

```
new_persons = [
    {'name': 'David', 'age': 28},
    {'age': 34, 'name': 'Jane'},
    {'name': 'Anna', 'city': 'Berlin'},
    {'name': 'Martin', 'age': 20, 'city':
'London'}
]

with open('output.csv', 'a', newline='') as f:
    writer = csv.DictWriter(f,
        fieldnames=['name', 'age'],
        extrasaction='ignore')
    writer.writerows(new_persons)
```

Пояснения

- У разных словарей разные ключи и их порядок
- Атрибут 'a' при открытии файла – новые строки будут дописаны в конце этого файла
- writeheader() не вызываем, чтобы избежать повторной записи строки с заголовками
- Атрибут extrasaction='ignore' функции DictWriter() позволяет игнорировать лишние ключи в записываемых словарях, если их нет в списке fieldnames

Работа с пропущенными значениями



В CSV-файлах пропущенные обозначаются как два разделителя подряд (обычно – две запятых подряд) или разделителем, после которого следует конец строки

Пример обработки пропущенных значений

```
with open('output.csv') as f:
    reader = csv.reader(f)
    good_rows = []
    for row in reader:
        # Проверяем, есть ли в строке пропущенные значения
        if any(field == '' for field in row):
            print("Пропущенные значения обнаружены, строка пропущена:",
row)
            continue
        good_rows.append(row)

print("\nСтроки с полными данными:")
for row in good_rows:
    print(*row)
```



ВОПРОСЫ





Недостатки модуля CSV

Недостатки модуля CSV

-  Отсутствие обработки пропущенных значений
-  Нет поддержки различных типов данных
-  Ограниченная обработка ошибок
-  Нет поддержки сложных структур данных
-  Ограниченные возможности фильтрации и агрегации данных

Важно



Для более продвинутой работы с CSV-файлами и данными в целом рекомендуется использовать специализированные библиотеки, например, модуль pandas.



ВОПРОСЫ





ЗАДАНИЯ



Выберите верный вариант ответа

1. Что будет выведено в результате выполнения следующего кода?

```
import csv

with open('data.csv', 'w', newline='')
as f:
    writer = csv.writer(f)
    writer.writerow(['name', 'age'])
    writer.writerow(['Alice', 30])

with open('data.csv') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

- a. ['name', 'age']
['Alice', 30]
- b. ['name', 'age']
['Alice', '30']
- c. name, age
Alice, 30
- d. ['Alice', '30']

Выберите верный вариант ответа

1. Что будет выведено в результате выполнения следующего кода?

```
import csv

with open('data.csv', 'w', newline='')
as f:
    writer = csv.writer(f)
    writer.writerow(['name', 'age'])
    writer.writerow(['Alice', 30])

with open('data.csv') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

- a. ['name', 'age']
['Alice', 30]
- b. ['name', 'age']
['Alice', '30']
- c. name, age
Alice, 30
- d. ['Alice', '30']



Выберите верный вариант ответа

2. Что делает метод `writerows()` в модуле `csv`?
- a. Записывает заголовки в CSV-файл
 - b. Записывает одну строку в CSV-файл
 - c. Записывает сразу несколько строк в CSV-файл
 - d. Читает все строки из CSV-файла



Выберите верный вариант ответа

2. Что делает метод `writerows()` в модуле `csv`?

- a. Записывает заголовки в CSV-файл
- b. Записывает одну строку в CSV-файл
- c. Записывает сразу несколько строк в CSV-файл
- d. Читает все строки из CSV-файла



Выберите верный вариант ответа

3. Что произойдёт, если передать в DictWriter словарь с лишними ключами и не указать `extrasaction='ignore'`?

- a. Лишние ключи будут проигнорированы
- b. Лишние ключи будут автоматически добавлены в заголовок
- c. Будет записано только совпадающее по ключам
- d. Возникнет ошибка `ValueError`



Выберите верный вариант ответа

3. Что произойдёт, если передать в DictWriter словарь с лишними ключами и не указать `extrasaction='ignore'`?

- a. Лишние ключи будут проигнорированы
- b. Лишние ключи будут автоматически добавлены в заголовок
- c. Будет записано только совпадающее по ключам
- d. Возникнет ошибка `ValueError`



Выберите верный вариант ответа

4. Почему при открытии файла для записи CSV рекомендуется указывать `newline=''`?
- a. Чтобы автоматически преобразовывать строки в числа
 - b. Чтобы не возникало лишних пустых строк между записями
 - c. Чтобы ускорить запись данных
 - d. Чтобы добавить заголовок в файл



Выберите верный вариант ответа

4. Почему при открытии файла для записи CSV рекомендуется указывать `newline=''`?

- a. Чтобы автоматически преобразовывать строки в числа
- b. Чтобы не возникало лишних пустых строк между записями
- c. Чтобы ускорить запись данных
- d. Чтобы добавить заголовок в файл

Выберите верный вариант ответа

5. Что произойдёт при выполнении следующего кода, если файл `data.csv` ещё не существует?

```
import csv
```

```
data = [
    {'name': 'Alice', 'age': 30},
    {'name': 'Bob', 'age': 25}
]
```

```
with open('data.csv', 'a', newline='')
as f:
    writer = csv.DictWriter(f,
fieldnames=['name', 'age'])
    writer.writerows(data)
```

- a. Файл будет создан и в него запишутся только данные, без заголовка
- b. Будет записан заголовок и данные
- c. Ничего не произойдёт — 'a' не работает с несуществующими файлами
- d. Произойдёт ошибка, так как `writeheader()` не был вызван

Выберите верный вариант ответа

5. Что произойдёт при выполнении следующего кода, если файл `data.csv` ещё не существует?

```
import csv
```

```
data = [
    {'name': 'Alice', 'age': 30},
    {'name': 'Bob', 'age': 25}
]
```

```
with open('data.csv', 'a', newline='')
as f:
    writer = csv.DictWriter(f,
fieldnames=['name', 'age'])
    writer.writerows(data)
```

- a. Файл будет создан и в него запишутся только данные, без заголовка
- b. Будет записан заголовок и данные
- c. Ничего не произойдёт — 'a' не работает с несуществующими файлами
- d. Произойдёт ошибка, так как `writeheader()` не был вызван



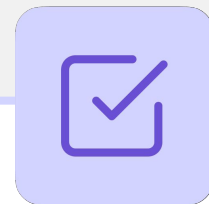
ВОПРОСЫ





Модуль statistics

Важно



Модуль `statistics` входит в стандартную библиотеку Python и используется для получения основных статистических характеристик числовых данных

Среднее арифметическое – mean()

Основные статистические функции



Пояснение

Обычное среднее значение: сумма всех чисел, делённая на их количество

Формула

$$\text{mean} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

Среднее геометрическое – `geometric_mean()`

Основные статистические функции



Пояснение

Уместно, когда усредняемые значения объединяются посредством умножения

Формула

$$\text{geometric mean} = \sqrt[n]{x_1 \cdot x_2 \cdot \dots \cdot x_n}$$

Среднее гармоническое – harmonic_mean()

Основные статистические функции



Пояснение

Используется, когда уместнее учитывать
обратные значения

Формула

$$\text{harmonic mean} = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \dots + \frac{1}{x_n}}$$

Мода – mode()

Основные статистические функции



Определение

Самое часто встречающееся значение в наборе данных

Пояснение

В случае нескольких мод — возвращает первую найденную. Для числовых и категориальных данных

Медиана – median()

Основные статистические функции



Определение

Значение, находящееся посередине
отсортированного ряда

Пояснение

Если чисел чётное количество —
берётся среднее из двух центральных.
Медиана устойчива к выбросам

Квантиль – quantiles()

Основные статистические функции



Определение

Делит упорядоченные данные на равные части

Пояснение

Например, квартиль — это деление на 4 части:

- 1-й квартиль (квантиль 25%) — значение, ниже которого лежит четверть данных
- 2-й квартиль — медиана
- 3-й квартиль (квантиль 75%) — значение, ниже которого лежит три четверти данных

Стандартное отклонение – stdev()

Основные статистические функции



Пояснение

Показывает, насколько значения в выборке отклоняются от среднего

Формула

$$\text{stdev} = \sqrt{\frac{1}{n-1} \sum (x_i - \bar{x})^2}$$

Дисперсия – variance()

Основные статистические функции



Пояснение

Квадрат стандартного отклонения.
Характеризует разброс данных относительно среднего

Формула

$$\text{variance} = \frac{1}{n-1} \sum (x_i - \bar{x})^2$$

Пример использования

```
import statistics

# Набор данных
data = [12, 15, 14, 10, 1, 18, 17, 15, 15, 14, 200]

# Средние значения
print("Арифметическое среднее:", statistics.mean(data))
print("Геометрическое среднее:", statistics.geometric_mean(data))
print("Гармоническое среднее:", statistics.harmonic_mean(data))

# Характеристики на отсортированном наборе
print("Отсортированный список:", sorted(data))
print("Медиана:", statistics.median(data))
print("Мода:", statistics.mode(data))

# Квантильные значения (делим на 4 части – квартиль)
print("Квартили:", statistics.quantiles(data, n=4))

# Измерения разброса
print("Стандартное отклонение:", statistics.stdev(data))
print("Дисперсия:", statistics.variance(data))
```



ВОПРОСЫ





ЗАДАНИЯ





Вычислите основные статистические характеристики списков чисел

```
data1 = [12, 15, 14, 10, 1, 18, 17, 15, 15, 14, 20]
data2 = [12, 15, 14, 10, 9, 18, 17, 15, 15, 14, 20]
data3 = [12, 15, 14, 10, 9, 18, 17, 15, 15, 14, 200]
```

Сравните результаты по всем четырем спискам. Проанализируйте, какие значения изменились, а какие нет. Объясните почему.



Ответ

Сильнее всего меняются:

- среднее арифметическое,
- среднее гармоническое
- меры разброса

– большое влияние на них оказывают сильно отклоняющиеся значения на краях отсортированного списка. Такие значения называются выбросами.

Мода, медиана и квартили слабо чувствительны к выбросам



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА

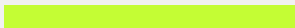


1. Чтение файла в список словарей



Прочитайте файл `sales.csv` в список словарей. Преобразуйте данные к корректным типам: строки, даты, числа. Вычислите все подходящие статистические характеристики каждого столбца.

2. Работа с JSON и CSV



Прочитайте файл `grades.json`. Запишите в csv-файл информацию по предметам Science, Math и Physics.



ДОМАШНЕЕ ЗАДАНИЕ



Домашнее задание

Работа с CSV-файлами

На лекции при выполнении практического задания 2 был получен файл `grades.csv`.

Напишите программу, которая:

1. Читает этот файл.
2. Создает на его основе три новых файла: `grades-Science.csv`, `grades-Math.csv`, `grades-Physics.csv`. В каждом из этих файлов только два столбца - имя и количество баллов по тому предмету, который указан в названии файла.
3. Создает четвертый файл `grades-info.csv`. В этом файле три строки (по названиям предметов) и столбцы со статистическими характеристиками оценок: среднее арифметическое, минимум, максимум, медиана, стандартное отклонение.

Заключение

